4 주차 학습노트

2022-16684 재료공학부 안휘준

인공지능의 면적 평가는 맞았을까

Talks 에서 인공지능은 점수 표시를 뺀 회로(No_Score)가 약 12,000~15,000 게이트라 ETRI 0.5um 칩에 들어가기 어렵다고 평가했다. 그리고 25 마리 외계인을 한 클럭에 한 마리씩 검사하고 곱셈을 시프트로 바꾼 코드를 주며 면적이 80% 이상 줄어든다고 답했다. 그런데 이 코드는 두 번 빌드에 실패했다. v_1 은 변수 선언 세 개가 빠졌고, v_2 는 3 비트 `loop_r` 가 0 에서 1 을 빼면 7 이 되어 반복문이 끝나지 않았다. 질문자가 지적한 이 오류들은 Verilator 로도 그대로 확인되었다.

```
hj@WSL: ~/space_invader_by_Gemini/Q.14_SC_Final_ETRI050/space_invaders_engine_glcd
$ verilator --lint-only -Wno-WIDTHTRUNC -Wno-WIDTHEXPAND -Wno-UNSIGNED 2_space_invaders_engine_glcd.v_1
%Error: line 301:18: Can't find definition of variable: 'c'
%Error: line 304:21: Can't find definition of variable: 't_invader_x'
%Error: line 305:21: Can't find definition of variable: 't_invader_y'
%Error: Exiting due to 3 error(s)
$ verilator --lint-only -Wno-WIDTHTRUNC -Wno-WIDTHEXPAND -Wno-UNSIGNED 3_space_invaders_engine_glcd.v_2
%Warning-SELRANGE: line 304:52: Extracting 6 bits from only 3 bit number
%Warning-SELRANGE: line 304:52: Selection index out of range: 5:0 outside 2:0
%Warning-INFINITELOOP: line 234:25: Infinite loop (condition always true)
%Error: Exiting due to 3 warning(s)
$ verilator --lint-only -Wno-WIDTHTRUNC -Wno-WIDTHEXPAND -Wno-UNSIGNED 4_space_invaders_engine_glcd.v_3
(오류 없음)
```

인공지능이 고친 세 코드를 Verilator 로 검사한 결과

Yosys 와 ETRI 표준 셀 라이브러리로 직접 합성해 숫자를 비교했다.

회로	인공지능의 예상	직접 합성한 결과
1 번 No_Score	12,000~15,000 게이트	셀 1,610 개, 0.524 mm² (NAND2 약 2,430 개)
4 번 v_3 (인공지능 개선판)	1,800~2,200 게이트, 80% 이상 감소	셀 1,579 개, 0.520 mm², 0.8% 감소

인공지능의 면적 예상과 실제 합성 결과

면적이 거의 줄지 않은 것은 `c*12` 같은 상수 곱셈을 합성 도구가 원래 시프트와 덧셈으로 바꾸기 때문이다. 또 원래 회로가 인공지능의 예상보다 5~6 배 작아서 줄어들 여지가 처음부터 적었다. 다만 v_3 로 같은 키 입력을 넣어 보니 게임은 0 번과 똑같이 동작했다. 순차 검사는 한 마리씩 25 클럭에 한 바퀴를 돌지만, 총알은 32,768 클럭에 한 칸만 움직이므로 충돌을 놓치지 않는다.

simulation 에서 고르는 다섯 RTL

make 가 고르게 하는 다섯 RTL 은 Q.13 최종본을 칩에 맞게 줄여 가는 과정이다. 0 번부터 좌표 출력(`lcd_x`, `lcd_y`) 대신 화소 값(`lcd_data`)과 화면 시작 신호(`v_sync`)만 내보내서 칩의 핀 수를 줄였다.

번호	무엇이 다른가	확인 결과
0 `.v.Score`	Q.13 최종본의 출력을 칩용으로 바꿈. 점수 표시 있음	빌드와 게임 모두 정상
1 `.v.No_Score`	점수 표시(숫자 글꼴 ROM, 점수 카운터)를 뺌	면적 16% 감소
2 `.v_1`	인공지능의 순차 검사판	선언 누락으로 빌드 실패
3 `.v_2`	빠진 선언을 추가함	무한 반복 경고로 빌드 실패
4 `.v_3`	`loop_r` 를 integer 로 바꿈	빌드 정상, 게임은 1 번과 같음

Q.14 simulation 에서 고를 수 있는 RTL

emulation 빌드와 동작 방식

Quartus Prime Lite 와 arduino-cli 를 설치해 Q.13 과 Q.14 의 세 단계 빌드를 모두 성공시켰다.

emulation 은 simulation 에서 고른 RTL 을 그대로 쓰므로, RTL 을 고르기 전에는 FPGA 빌드가 실패했다.

```
hj@WSL: ~/space_invader_by_Gemini/Q.14_SC_Final_ETRI050/emulation
$ make build_mi        # 1단계: Pico 펌웨어 (arduino-cli)
arduino-cli compile --clean --fqbn rp2040:rp2040:rpipico Espace_invaders_engine_glcd_CA ...
Sketch uses 72100 bytes (3%) of program storage space. Maximum is 2093056 bytes.
Global variables use 11696 bytes (4%) of dynamic memory.
$ make build_trans     # 2단계: FPGA 회로 (quartus_sh)
quartus_sh -t space_invaders_engine_glcd_wrapper.tcl
Info (293000): Quartus Prime Full Compilation was successful. 0 errors, 31 warnings
Info: Quartus Prime Shell was successful. 0 errors, 31 warnings
$ make build           # 3단계: PC 쪽 테스트벤치 (clang++)
clang++ -I/opt/systemc/include -L/opt/systemc/lib ...
-> 실행 파일 sc_space_invaders_engine_glcd_TB (90272 bytes)
```

Q.14 emulation 디렉토리의 세 단계 빌드

PC 테스트벤치는 시뮬레이션 것을 그대로 쓰고, 회로 모델 자리에만 FPGA 와 통신하는 모델을 끼운다.

클럭이 오르내릴 때마다 PC 가 입력 1 바이트를 보내고 출력 1 바이트를 받으므로, 한 화면(8,192 클럭)에 16,384 번을 주고받는다. 클럭 자체도 PC 가 보내는 한 비트라서 FPGA 는 PC 가 보내 준 만큼만 진행한다.

make 한 줄 뒤에서 일하는 도구들

디렉토리	make 타깃	실제로 부르는 도구
simulation	`lint`, `build`, `run`	verilator(RTL 을 C++로 변환) → clang++ 컴파일 → 실행 파일
emulation	`build_mi`, `upload_mi`	arduino-cli (Pico 펌웨어)
	`build_trans`, `config_trans`	quartus_sh, quartus_cpf, openFPGALoader (FPGA 회로)
	`build`, `run`	clang++ (SystemC 테스트벤치)
ETRI050	`synthesize ~ lvs`	qflow → yosys, graywolf, OpenSTA, qrouter, magic, netgen

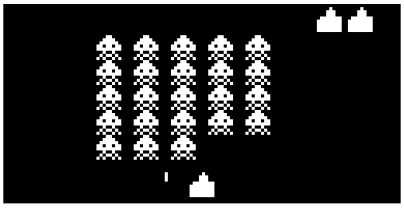
디렉토리와 make 타깃별 도구

꺼 둔 경고 속에 숨어 있던 버그

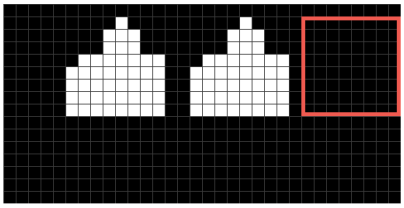
make 가 Verilator 에 주는 옵션을 살펴보다가 `-Wno-WIDTHTRUNC` 처럼 경고를 끄는 옵션이 눈에 띄었다. 이 옵션을 빼고 v_3 를 검사하자 `scan_x < 7'd128` 에서 경고가 나왔다. 7 비트는 127 까지만 담을 수 있어 128 이 0 으로 잘리고, '0 보다 작다'는 조건은 늘 거짓이 된다. 실제로 목숨이 3 개인데 화면에는 아이콘이 2 개만 그려졌다. 같은 코드가 Q.13 최종본부터 다섯 RTL 모두에 들어 있었다.

```
hj@WSL: ~/space_invader_by_Gemini/Q.14_SC_Final_ETRI050/space_invaders_engine_glcd
$ verilator --lint-only -Wall 4_space_invaders_engine_glcd.v_3 | grep 128
%Warning-WIDTHTRUNC: line 18:32: Value too large for 7 bit number: 128
%Warning-WIDTHTRUNC: line 323:70: Value too large for 7 bit number: 128
```

경고를 끄는 옵션을 빼고 검사한 결과



4번 v_3 게임 화면



오른쪽 위 확대: 세 번째 목숨 자리

목숨 3 개 중 세 번째 아이콘이 그려지지 않는 화면

여기서 의문이 하나 생겼다. 18 번 줄의 `SCREEN_WIDTH = 7'd128` 도 0 으로 잘리지만, `SCREEN_WIDTH - 1` 처럼 빼는 데에만 쓰여 127 이 되므로 우연히 맞게 동작한다. 경고를 끄면 이렇게 '우연히 맞는 코드'와 '정말 틀린 코드'가 구별되지 않는다. 시뮬레이션에서는 화면을 보고 알아챌 수 있었지만, 칩으로 만든 뒤에는 고칠 수 없다. 그렇다면 ETRI050 에서 합성하기 전에는 꺼 두었던 경고를 다시 켜고 하나씩 확인하는 단계가 필요하지 않을까?

느낀 점

인공지능은 면적이 80% 넘게 줄어든다고 자신 있게 말했지만, 직접 합성하니 1%도 줄지 않았다. 인공지능이 고친 코드도 두 번 빌드에 실패했다. 인공지능의 답은 검사 도구로 확인해야 쓸 수 있다는 것을 다시 느꼈다.

make 를 실행만 했다면 목숨 아이콘 문제는 찾지 못했을 것이다. 각 단계가 어떤 도구에 어떤 옵션을 주는지 들여다보니 그 옵션이 무엇을 숨기고 있는지도 보였다.